

A COMPLETE TECHNICAL COURSE

PostgreSQL Transactions & Locking

A comprehensive, query-driven guide to ACID, transaction control, isolation levels, MVCC, row and table locking, deadlocks, and production diagnostics.

Inside: ACID • Transaction Control • Isolation Levels • MVCC • Row & Table Locks • Deadlocks • Diagnostics

1

Foundations of Transactions

What a transaction is, and why every serious database needs one

What Is a Database Transaction?

1 A logical unit of work

One or more SQL statements — reads and writes — that PostgreSQL treats as a single, indivisible operation.

2 All-or-nothing execution

Every statement inside the transaction succeeds and is made permanent together, or none of them take effect at all.

3 Bounded by explicit or implicit boundaries

In PostgreSQL, an explicit transaction begins with `BEGIN` (or `START TRANSACTION`) and ends with `COMMIT` or `ROLLBACK`.

4 The unit the database guarantees

PostgreSQL never promises correctness for an individual SQL statement in isolation — the transaction is the real unit of correctness.

Note: Even a single `UPDATE` statement runs inside an implicit, single-statement transaction — PostgreSQL never executes SQL outside of a transaction context.

Why Databases Need Transactions

1 Multi-step operations must not be left half-done

A funds transfer debits one account and credits another — a crash between the two steps must not lose money.

2 Concurrent users would otherwise corrupt each other's work

Without isolation guarantees, two simultaneous updates to the same row can silently overwrite one another.

3 Hardware and software fail mid-operation

Power loss, process crashes, and network failures can interrupt a write at any point — the database must recover cleanly.

4 Business rules span multiple rows and tables

An order isn't valid unless the order row, its line items, and the inventory decrement all succeed together.

Note: Transactions convert 'many things must happen together, reliably, under concurrency' into a guarantee the application can depend on.

Real-World Transaction Examples



Bank Transfer

Debit account A
Credit account B
Both or neither happens
Never a partial transfer



E-Commerce Checkout

Insert order row
Insert order line items
Decrement stock
Charge payment reference



Inventory Adjustment

Reserve stock for order
Release stock on cancel
Must stay consistent
Across concurrent orders



Seat Reservation

Check seat availability
Insert booking row
Mark seat unavailable
No double-booking

2

The ACID Properties

The four guarantees every PostgreSQL transaction provides

Atomicity — All or Nothing

1 Definition

Every statement in a transaction commits together, or the entire transaction is rolled back as if it never ran.

2 How PostgreSQL enforces it

Uncommitted changes are only ever visible to the transaction that made them; ROLLBACK discards every row version it created.

3 Failure handling

If any statement raises an error, the transaction is marked aborted and every subsequent statement is rejected until ROLLBACK.

4 Example

A transaction that debits one account and crashes before crediting the second leaves the database exactly as it was before BEGIN.

Consistency — Valid State to Valid State

1 Definition

A transaction moves the database from one state that satisfies all rules to another state that also satisfies all rules.

2 Enforced by constraints

PRIMARY KEY, UNIQUE, CHECK, NOT NULL, and FOREIGN KEY constraints are validated before a transaction is allowed to commit.

3 Application responsibility too

The database enforces declared constraints; business-rule consistency beyond those constraints is the application's responsibility.

4 Example

A CHECK (balance $\geq$ 0) constraint guarantees no committed transaction can ever leave an account with a negative balance.

Isolation — Concurrent Transactions Don't Interfere

1 Definition

Each transaction behaves as though it were running alone, even when many transactions execute concurrently.

2 Degree is configurable

PostgreSQL lets you choose how strictly isolation is enforced via the isolation level: Read Committed, Repeatable Read, or Serializable.

3 Implemented primarily through MVCC

PostgreSQL gives each transaction a consistent snapshot of the data rather than blocking every reader behind every writer.

4 The trade-off

Stricter isolation prevents more anomalies but increases the chance of blocking or serialization failures under contention.

Durability — Committed Means Permanent

1 Definition

Once COMMIT returns successfully, the transaction's changes survive any subsequent crash, power loss, or restart.

2 How PostgreSQL enforces it

Write-Ahead Log (WAL) records are flushed to durable storage before COMMIT is acknowledged to the client.

3 Recovery on restart

After a crash, PostgreSQL replays the WAL to reconstruct every committed change that had not yet reached the data files.

4 Tunable trade-off

`synchronous_commit` can be relaxed for throughput, trading a small durability window for lower commit latency.

ACID in Action — A Bank Transfer

A transaction that debits one account and credits another, protected end-to-end:



```
BEGIN;
```

```
UPDATE accounts SET balance = balance - 250 WHERE id = 1;
```

```
UPDATE accounts SET balance = balance + 250 WHERE id = 2;
```

```
-- If either statement violates a CHECK constraint, or the
```

```
-- application detects a problem, it issues:
```

```
-- ROLLBACK;
```

```
COMMIT;
```

Atomicity: both UPDATES take effect together, or neither does.

Consistency: a CHECK (balance $\geq$ 0) constraint blocks any commit that would overdraw an account.

Isolation: concurrent readers never see the intermediate state where only one leg has run.

Durability: once COMMIT returns, the transfer survives a server crash one millisecond later.

3

How PostgreSQL Implements ACID

A short primer on WAL and MVCC before we go deeper

Write-Ahead Logging (WAL) and Durability

1 Log first, data files later

Every change is first recorded in the Write-Ahead Log; the actual table and index pages are updated afterward, often lazily.

2 COMMIT waits on the log, not the data

PostgreSQL only reports a transaction as committed once its WAL records are safely flushed to disk.

3 Crash recovery replays the log

On restart after a crash, PostgreSQL replays WAL from the last checkpoint to restore every committed change.

4 WAL also powers replication

Streaming replication and point-in-time recovery both work by shipping and replaying the same WAL stream.

Atomicity and Consistency via the Transaction Log

1 Transaction IDs (XIDs)

Every transaction gets a unique, monotonically increasing transaction ID that tags every row version it creates.

2 Commit status is the source of truth

The commit log (`pg_xact`) records whether each XID ultimately committed or aborted — that status decides visibility.

3 Rollback needs no undo of data pages

Because uncommitted row versions are simply marked with an aborted XID, ROLLBACK is nearly instantaneous — no data is rewritten.

4 Constraint checks gate commit

Deferred constraints (like some foreign keys) are validated just before COMMIT, not at each individual statement.

WAL's Job vs. MVCC's Job

Write-Ahead Log (WAL)

- Guarantees durability after COMMIT
- Enables crash recovery
- Powers physical & logical replication
- Records every physical change, in order
- Is largely invisible to application code

MVCC (Multi-Version Concurrency Control)

- Guarantees isolation between concurrent transactions
- Lets readers proceed without blocking writers
- Keeps multiple versions of each row
- Decides which row version each query can see
- Directly shapes how you write concurrent SQL

4

Transaction Control Commands

BEGIN, COMMIT, ROLLBACK, SAVEPOINT, and everything around them

BEGIN / START TRANSACTION

Opens an explicit transaction block. BEGIN and START TRANSACTION are exact synonyms in PostgreSQL.



```
BEGIN;  
-- equivalently:  
START TRANSACTION;  
  
-- optional mode clauses:  
BEGIN ISOLATION LEVEL REPEATABLE READ;  
BEGIN READ ONLY;  
BEGIN READ WRITE, DEFERRABLE;
```

Everything after BEGIN runs inside one transaction block until COMMIT or ROLLBACK.

Calling BEGIN again while already inside a transaction produces a harmless warning.

Mode clauses (isolation level, READ ONLY, DEFERRABLE) can be set right on BEGIN.

COMMIT — Making Changes Permanent

Ends the current transaction successfully, making all its changes visible to other sessions.



```
BEGIN;  
  INSERT INTO orders (customer_id, total) VALUES (42, 199.99);  
  UPDATE inventory SET qty = qty - 1 WHERE sku = 'ABC-100';  
COMMIT;  
  
-- PostgreSQL 12+ synonym:  
COMMIT WORK;  
COMMIT AND CHAIN; -- commit, then immediately start a new tx
```

COMMIT only returns once the WAL for the transaction is durably flushed.

After COMMIT, changes are visible to any new snapshot taken by other sessions.

COMMIT AND CHAIN starts the next transaction with the same isolation properties.

ROLLBACK — Discarding a Transaction

Ends the current transaction and undoes every change made since `BEGIN`.



```
BEGIN;  
  UPDATE accounts SET balance = balance - 1000000 WHERE id = 1;  
  -- application logic detects a problem...  
ROLLBACK;  
  
-- Row versions created by the aborted transaction are simply  
-- marked dead; no data-page rewrite is needed.
```

ROLLBACK is fast: PostgreSQL marks the transaction's row versions as aborted rather than rewriting pages.

Any error inside a transaction block forces it into an aborted state until an explicit ROLLBACK.

ROLLBACK TO SAVEPOINT (next slides) allows a partial, more surgical undo.

Autocommit Mode — Implicit Transactions

1 Every statement runs in a transaction

PostgreSQL never executes SQL outside a transaction — a bare statement gets its own single-statement transaction automatically.

2 Autocommit is the client default

psql and most drivers commit immediately after each statement unless you explicitly issue BEGIN.

3 BEGIN suspends autocommit

Once you issue BEGIN, statements accumulate in the open transaction until you COMMIT or ROLLBACK.

4 Drivers can disable autocommit

Most client libraries (psycopg2, JDBC, etc.) offer a connection-level setting to turn autocommit off entirely.

Note: A single 'UPDATE ... WHERE id = 1;' with no BEGIN is really its own implicit BEGIN; ...; COMMIT; under the hood.

Autocommit: A Single Statement Transaction

Without an explicit BEGIN, this statement is its own complete transaction:



```
UPDATE accounts SET balance = balance + 50 WHERE id = 7;  
-- Equivalent to:  
BEGIN;  
  UPDATE accounts SET balance = balance + 50 WHERE id = 7;  
COMMIT;
```

If the UPDATE fails partway (e.g., a constraint violation), it is rolled back automatically.

No other statement can be grouped with it — each new statement is its own transaction.

For multi-statement atomicity, an explicit BEGIN ... COMMIT block is required.

SAVEPOINT — A Checkpoint Within a Transaction

Creates a named point you can later roll back to, without discarding the whole transaction.



```
BEGIN;  
  INSERT INTO orders (id, customer_id) VALUES (501, 42);  
  
  SAVEPOINT before_items;  
  INSERT INTO order_items (order_id, sku) VALUES (501, 'BAD-SKU');  
  -- error: foreign key violation
```

A SAVEPOINT can be created at any point inside an open transaction block.

Multiple savepoints can be nested, each with its own unique name.

A savepoint does not commit anything — it is purely a rollback anchor.

ROLLBACK TO SAVEPOINT — Partial Undo

Undoes only the statements executed after the named savepoint, keeping the transaction open.



```
BEGIN;  
INSERT INTO orders (id, customer_id) VALUES (501, 42);  
SAVEPOINT before_items;  
  INSERT INTO order_items (order_id, sku) VALUES (501, 'BAD-SKU');  
ROLLBACK TO SAVEPOINT before_items;  
  
INSERT INTO order_items (order_id, sku) VALUES (501, 'GOOD-SKU');  
COMMIT;
```

The failed order_items insert is undone; the orders insert from before the savepoint survives.

The transaction returns to a healthy state and can continue accepting new statements.

Without a savepoint, that earlier error would have aborted the entire transaction.

RELEASE SAVEPOINT — Discarding the Checkpoint

Releases a savepoint you no longer need, without undoing anything it protected.



```
BEGIN;  
  SAVEPOINT step1;  
  UPDATE inventory SET qty = qty - 1 WHERE sku = 'ABC-100';  
  RELEASE SAVEPOINT step1;  
  -- step1 no longer exists; its changes remain part of the tx  
COMMIT;
```

RELEASE SAVEPOINT merges that savepoint's changes into the surrounding transaction.

It frees the resources PostgreSQL used to track the savepoint.

Releasing is optional — unreleased savepoints are cleaned up automatically at COMMIT or ROLLBACK.

Nested Savepoints in Practice

Savepoints can be stacked to protect increasingly granular pieces of work:



```
BEGIN;  
SAVEPOINT sp1;  
  INSERT INTO log(msg) VALUES ('step 1 ok');  
  
  SAVEPOINT sp2;  
    INSERT INTO log(msg) VALUES ('step 2 failing');  
    -- error occurs here  
  ROLLBACK TO SAVEPOINT sp2; -- undo just step 2  
  
RELEASE SAVEPOINT sp1;  
COMMIT;
```

Rolling back to an inner savepoint leaves outer savepoints and their work untouched.

This pattern underlies how many ORMs implement nested 'atomic blocks'.

Each savepoint adds a small amount of overhead — avoid deep nesting in hot code paths.

Transaction States in PostgreSQL

1 Active / In Progress

The normal state — statements are being executed and their effects are visible only within this transaction.

2 Aborted

Entered after any statement error; PostgreSQL rejects every further statement except ROLLBACK, ROLLBACK TO SAVEPOINT, or COMMIT (which behaves like ROLLBACK).

3 Committed

The terminal success state — all changes are permanent and the transaction ID is recorded as committed.

4 `psql` shows the state

`psql`'s prompt changes to a `!` when the current transaction is aborted, e.g. `'mydb=*!#'`.

Note: "current transaction is aborted, commands ignored until end of transaction block" is the classic sign of this state.

Errors Abort the Whole Transaction

Without a savepoint, one bad statement poisons everything after BEGIN:



```
BEGIN;  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
UPDATE accounts SET balance = balance + 100 WHERE id = 999999;  
-- no such row is fine, but suppose a CHECK fails instead:  
-- ERROR: new row violates check constraint  
  
SELECT 1; -- ERROR: current transaction is aborted  
COMMIT;  -- has no effect; behaves like ROLLBACK
```

PostgreSQL refuses to run further statements once a transaction is aborted.

The only valid next commands are ROLLBACK or ROLLBACK TO an earlier SAVEPOINT.

Wrapping risky statements in a SAVEPOINT is the standard way to keep the transaction usable.

COMMIT AND CHAIN & Session Defaults

PostgreSQL 12+ lets you commit and immediately reopen a transaction with the same properties:



```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

```
-- ... work ...
```

```
COMMIT AND CHAIN;
```

```
-- new transaction starts, still REPEATABLE READ
```

```
-- ... more work ...
```

```
COMMIT;
```

```
ROLLBACK AND CHAIN; -- same idea, but discarding the tx
```

Useful for long batch jobs that need many short transactions with identical settings.

Avoids the small overhead and race window of a separate BEGIN after COMMIT.

default_transaction_isolation (a session setting) supplies the default when BEGIN omits it.

Transaction Control Command Reference

Command	Purpose	Notes
BEGIN / START TRANSACTION	Open an explicit transaction block	Exact synonyms; accepts mode clauses
COMMIT / COMMIT WORK	End the transaction, keep changes	Waits for WAL flush before returning
ROLLBACK	End the transaction, discard changes	Fast — no data pages rewritten
SAVEPOINT name	Mark a rollback anchor	Can be nested
ROLLBACK TO SAVEPOINT name	Undo back to that anchor	Transaction stays open
RELEASE SAVEPOINT name	Forget an anchor, keep its work	Optional cleanup
COMMIT AND CHAIN	Commit, then reopen with same settings	PostgreSQL 12+
SET TRANSACTION ...	Set isolation level / access mode	Must be first statement in the tx

5

Introduction to Isolation Levels

The SQL standard's four levels, and what each one promises

The SQL Standard's Four Isolation Levels

1 Read Uncommitted

The weakest level — in the standard, transactions may see uncommitted changes from other transactions (dirty reads).

2 Read Committed

A transaction only ever sees data committed before each statement begins. PostgreSQL's default.

3 Repeatable Read

A transaction sees a single consistent snapshot for its entire duration — the same query always returns the same rows.

4 Serializable

The strongest level — transactions behave as if executed one at a time, in some serial order, with no interleaving effects at all.

Note: The standard defines these levels by which anomalies they permit — not by how a database must implement them internally.

Standard Isolation Levels vs. Permitted Anomalies

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	Possible	Possible	Possible
Read Committed	Not Possible	Possible	Possible
Repeatable Read	Not Possible	Not Possible	Possible
Serializable	Not Possible	Not Possible	Not Possible

Note: This table describes the SQL standard. PostgreSQL's actual behavior — stricter in places — is covered in Section 7.

6

Concurrency Anomalies

The specific ways concurrent transactions can interfere with each other

Four Classic Read Phenomena



Dirty Read

Reads another tx's uncommitted change
That change may later be rolled back
You've now read data that never existed



Non-Repeatable Read

Same row read twice
Value differs between reads
Because another tx committed between them



Phantom Read

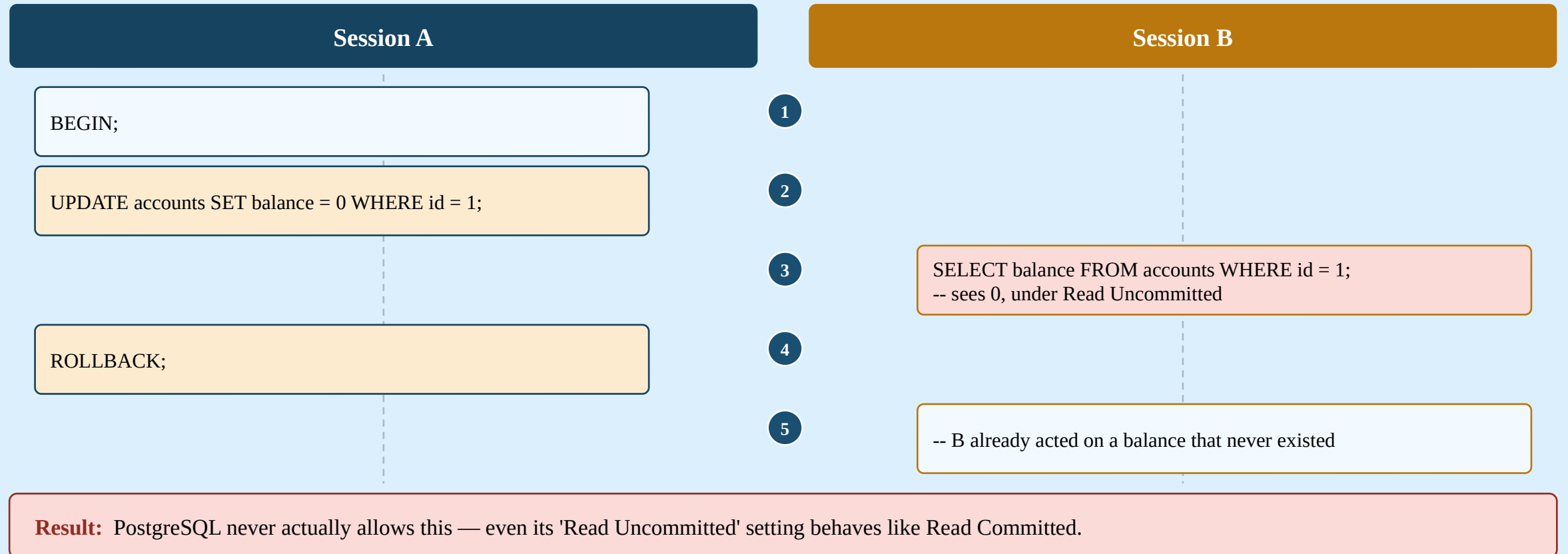
Same filtered query re-run
New rows appear or vanish
Because of concurrent inserts/deletes



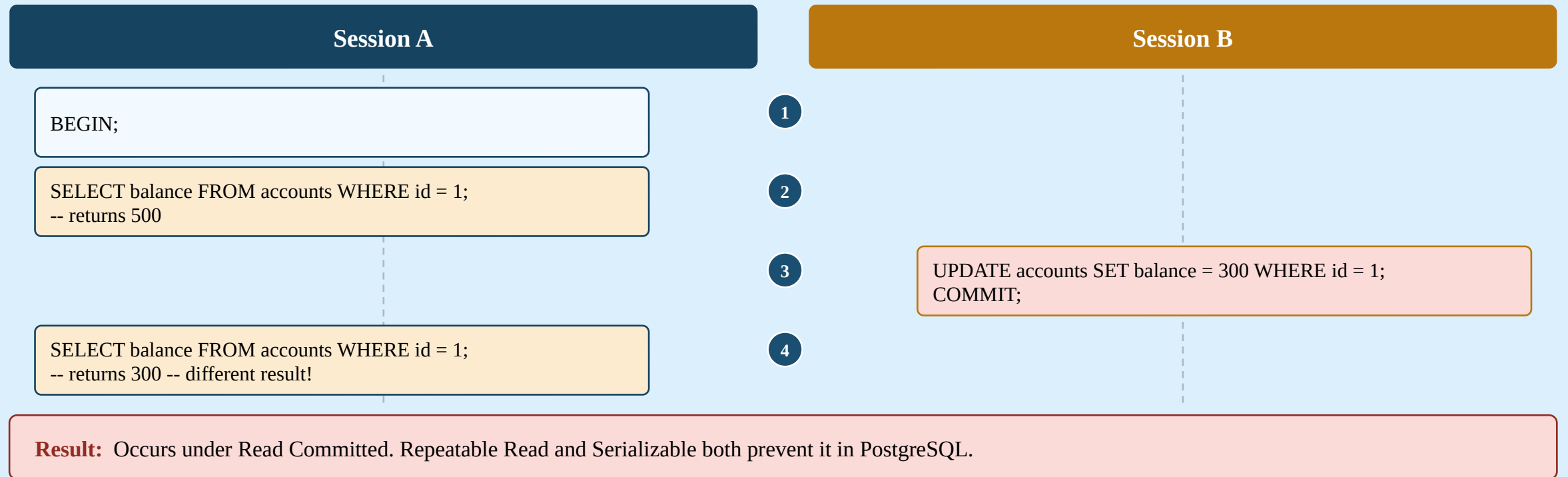
Lost Update

Two transactions read-modify-write
One overwrites the other's write
The first change is silently lost

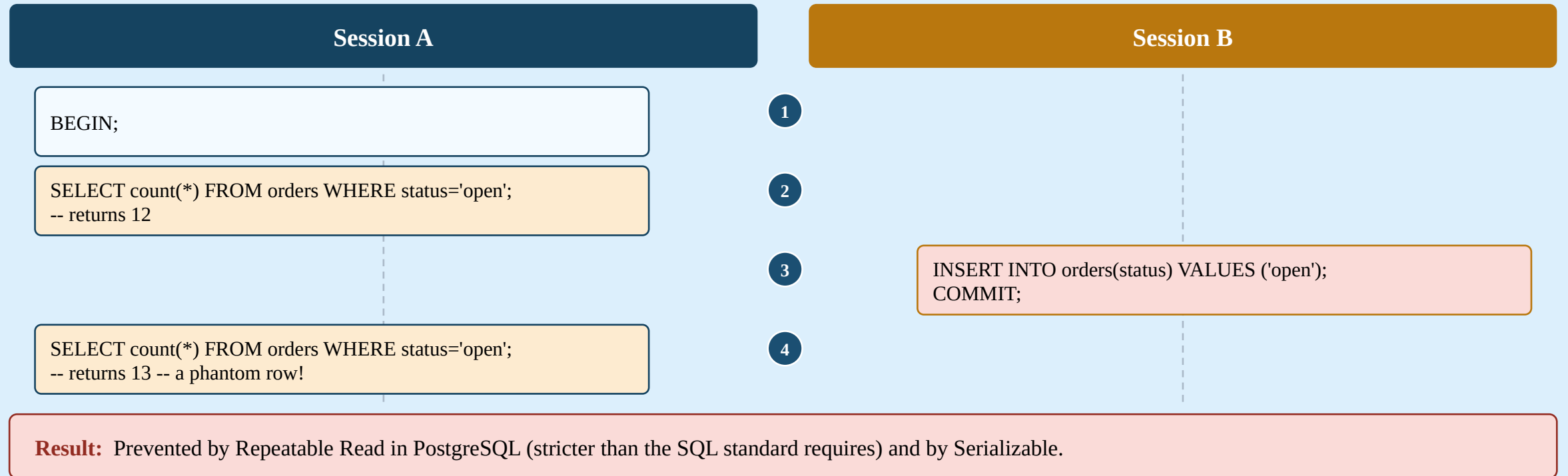
Dirty Read — Reading Uncommitted Data



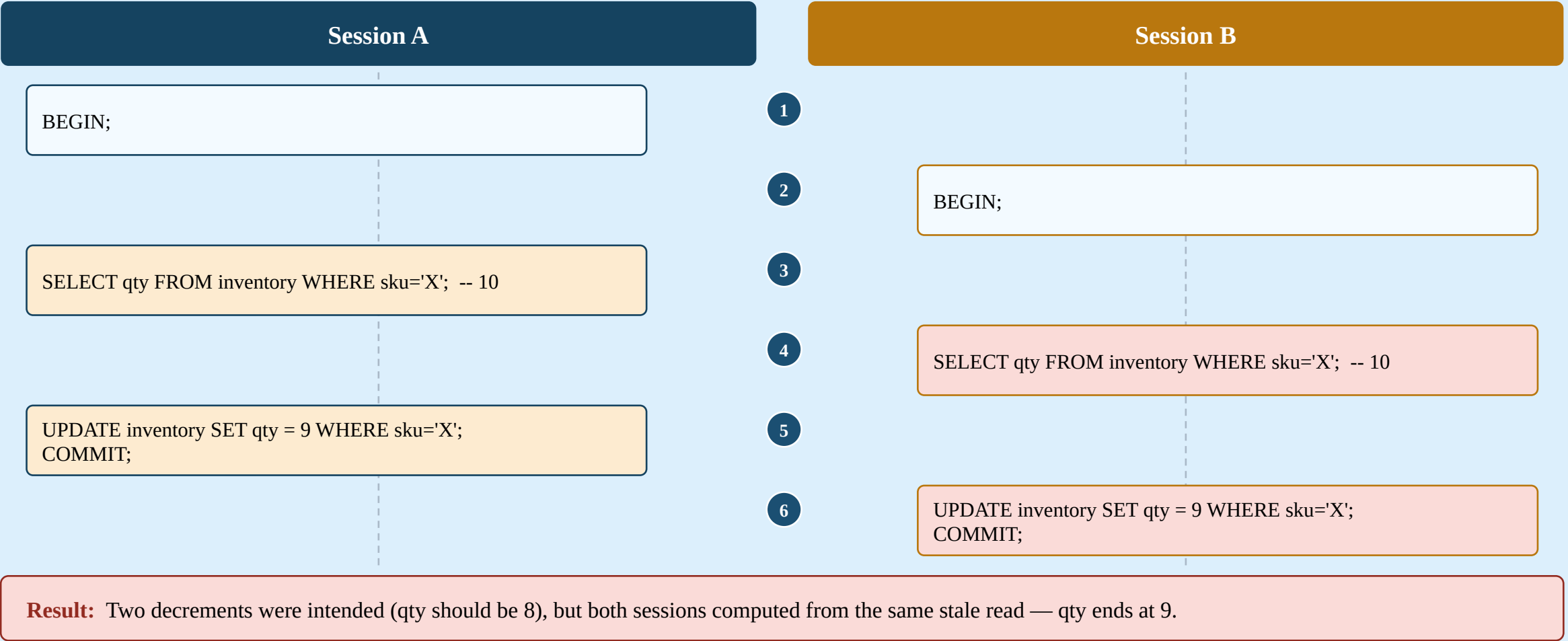
Non-Repeatable Read — A Value Changes Mid-Transaction



Phantom Read — New Rows Appear



Lost Update — One Write Silently Overwrites Another



Write Skew — A Subtler Anomaly

1 Definition

Two transactions each read overlapping data, each makes a decision based on what they read, and both commit — but together they violate a rule neither one violated alone.

2 Not caught by Repeatable Read

Because each transaction only writes rows the other didn't write, ordinary snapshot isolation does not detect the conflict.

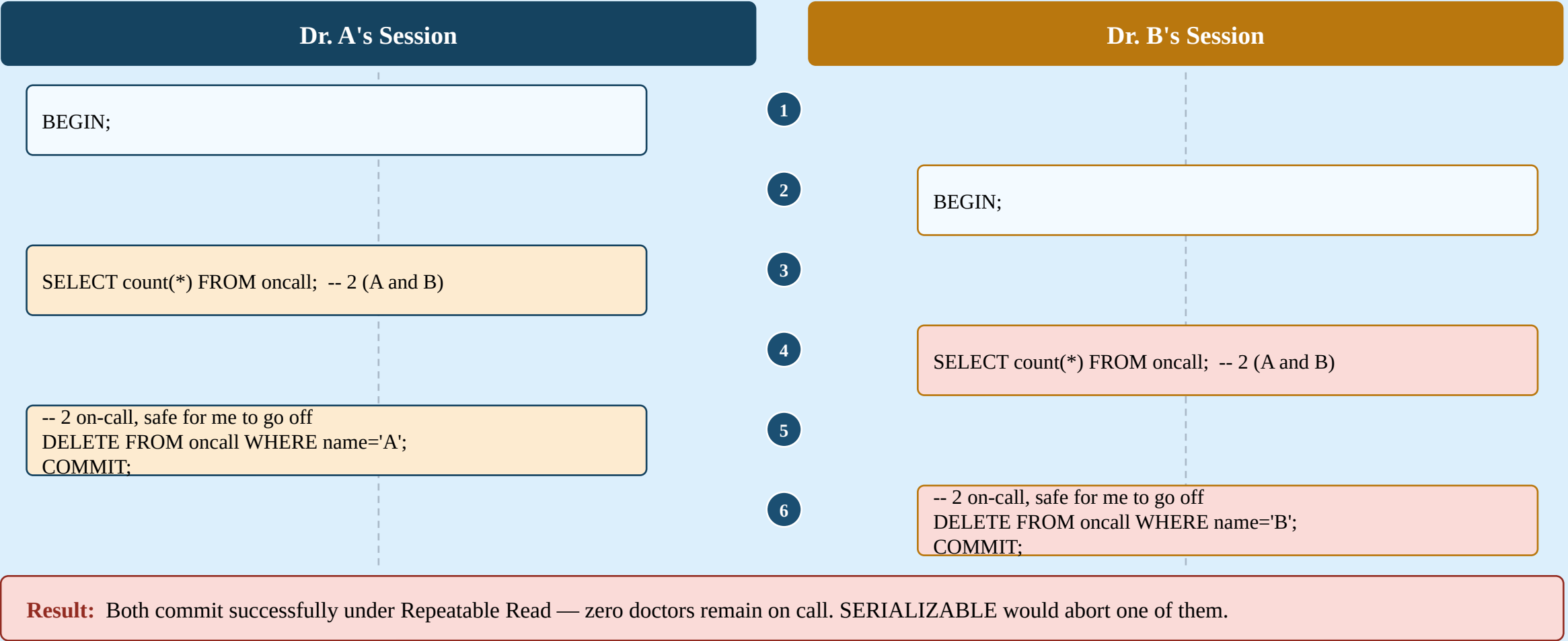
3 Only Serializable prevents it

PostgreSQL's Serializable Snapshot Isolation (SSI) specifically detects and aborts write-skew scenarios.

4 Classic example

An on-call rule requires at least one doctor on duty; two of the last two doctors each independently go off-call, believing the other is still on.

Write Skew Example — On-Call Doctors



7

Isolation Levels in PostgreSQL

How Read Committed, Repeatable Read, and Serializable actually behave

Read Committed — The Default

1 Snapshot per statement

Every individual statement gets a fresh snapshot, seeing all data committed before that statement began.

2 No dirty reads

A statement never sees another transaction's uncommitted changes — PostgreSQL guarantees this even though the standard permits it.

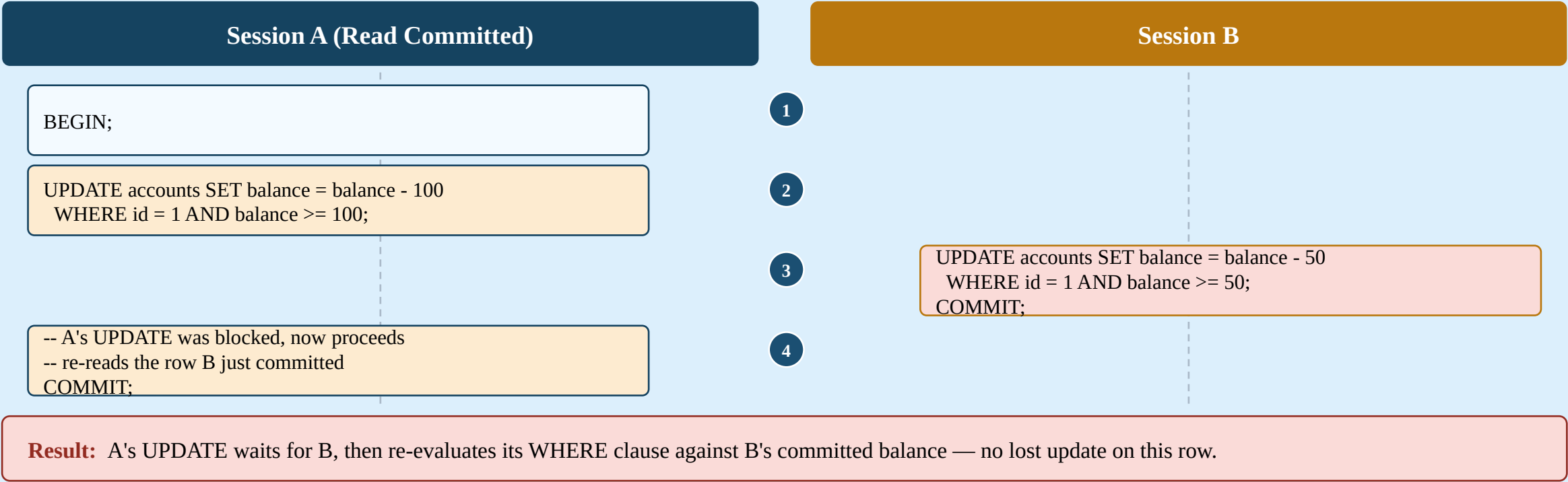
3 Non-repeatable reads are possible

Two SELECTs in the same transaction can return different results if another transaction commits in between.

4 UPDATE/DELETE re-check on conflict

If a row was changed by a concurrent committed transaction, Read Committed re-evaluates the WHERE clause against the new row version.

Read Committed in Action



Repeatable Read

1

Snapshot per transaction

The entire transaction sees one consistent snapshot, taken at the time of its first query — not refreshed per statement.

2

No dirty or non-repeatable reads

The same SELECT always returns the same rows and values for the life of the transaction.

3

No phantom reads (PostgreSQL is stricter than the standard)

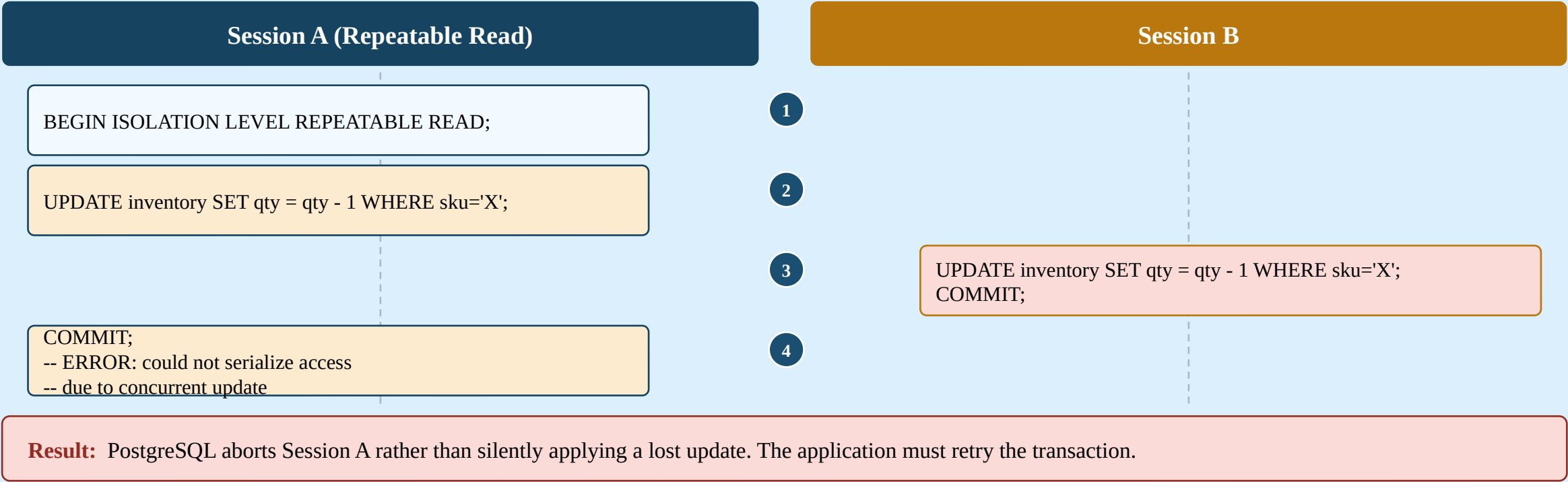
Unlike the SQL standard's minimum, PostgreSQL's Repeatable Read also blocks phantom rows.

4

Concurrent update conflicts raise an error

If this transaction tries to UPDATE a row another transaction already committed a change to, PostgreSQL raises a serialization failure (SQLSTATE 40001).

Repeatable Read in Action



Serializable (Serializable Snapshot Isolation)

1 The strongest guarantee

Transactions behave as if run one at a time in some serial order — including write-skew scenarios that Repeatable Read misses.

2 Built on Repeatable Read plus conflict tracking

PostgreSQL's SSI monitors read/write dependencies between concurrent Serializable transactions to detect unsafe patterns.

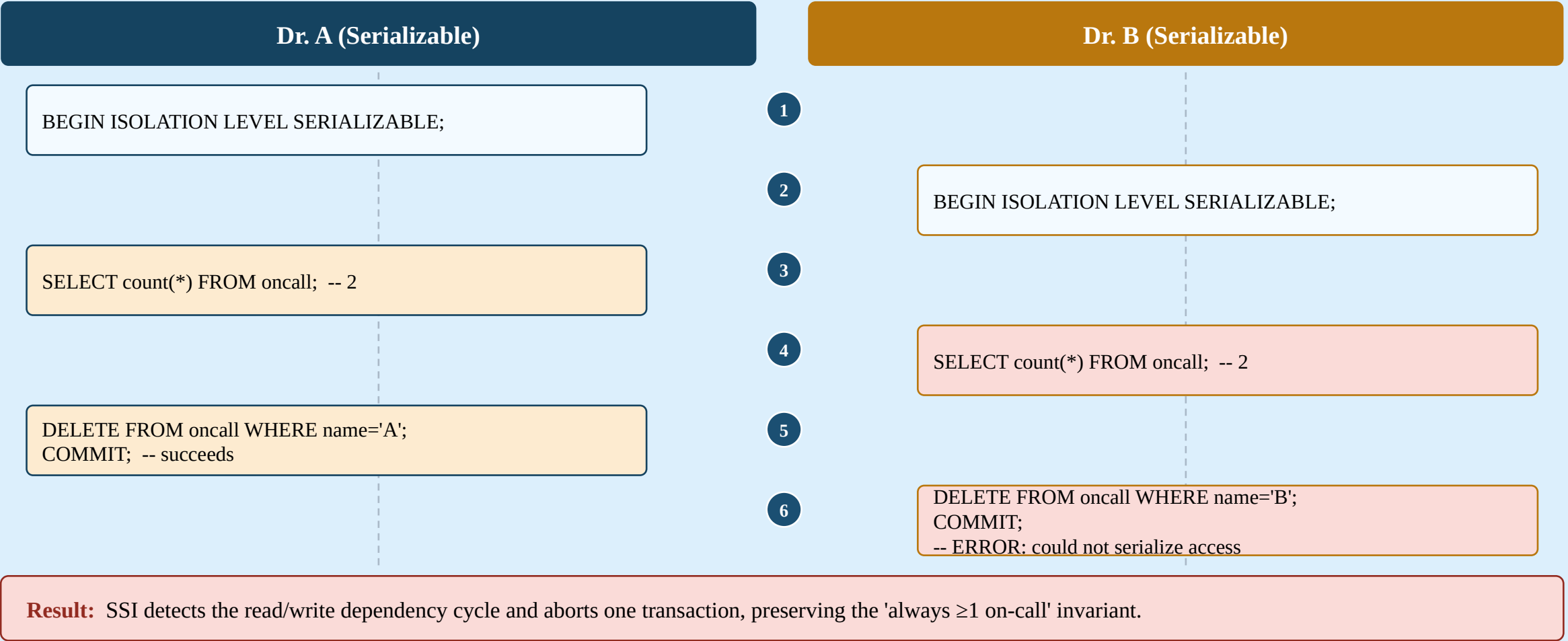
3 May abort with a serialization failure

When SSI detects a cycle that could produce a non-serializable outcome, it aborts one of the involved transactions.

4 Requires application-level retry logic

Any code using `SERIALIZABLE` must be prepared to catch `SQLSTATE 40001` and simply re-run the transaction.

Serializable in Action — Catching Write Skew



PostgreSQL Isolation Levels vs. Anomalies

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Write Skew
Read Committed	Not Possible	Possible	Possible	Possible
Repeatable Read	Not Possible	Not Possible	Not Possible	Possible
Serializable	Not Possible	Not Possible	Not Possible	Not Possible

Note: PostgreSQL implements 'Read Uncommitted' as an alias for Read Committed — true dirty reads never occur.

Choosing the Right Isolation Level

- 1 Read Committed — the default, for most workloads**
Good performance, intuitive behavior, and correct for the vast majority of single-statement or simple operations.
- 2 Repeatable Read — for multi-step reports and consistency-sensitive reads**
Use when a transaction must see one consistent view across several queries, e.g. generating a financial report.
- 3 Serializable — for correctness-critical invariants**
Use when business rules span multiple rows and must never be violated, even by rare interleavings — accept the retry cost.
- 4 Rule of thumb**
Start with Read Committed; escalate only where you can name the specific anomaly you're protecting against.

8

MVCC: Multi-Version Concurrency Control

The mechanism that makes PostgreSQL's isolation levels possible

What Is MVCC?

1 Multiple versions, one table

Instead of overwriting a row in place, an UPDATE inserts a brand-new row version and marks the old one as superseded.

2 Readers never block writers

A SELECT reads whichever row version was visible to its snapshot — it never needs to wait for a concurrent writer's lock.

3 Writers never block readers

Because readers don't take row locks for plain SELECTs, writers are never blocked by ongoing reads either.

4 The trade-off: old versions need cleanup

Dead row versions accumulate until VACUUM reclaims the space — this is the cost MVCC pays for lock-free reads.

Hidden System Columns — xmin, xmax, ctid

1 xmin

The transaction ID that created this row version. A row is visible only if xmin's transaction has committed (and isn't in your snapshot's future).

2 xmax

The transaction ID that deleted or updated-away this row version — zero means the row hasn't been superseded yet.

3 ctid

The physical location (page, offset) of this row version on disk; it changes every time the row is updated.

4 These columns are always there

Every table implicitly has xmin, xmax, ctid, and a few others — they just aren't shown unless explicitly selected.

Inspecting xmin/xmax Directly

You can query PostgreSQL's system columns like any other column:



```
SELECT xmin, xmax, ctid, balance
FROM accounts
WHERE id = 1;
```

```
-- xmin | xmax | ctid | balance
-- -----+-----+-----+-----
-- 48213 | 0 | (0,7) | 500
```

xmax = 0 means no transaction has yet superseded this row version.

After an UPDATE, the old row gets a non-zero xmax and a new row version appears with a fresh xmin.

This is invaluable for debugging visibility and bloat issues directly.

Snapshots and Row Visibility Rules

1 A snapshot is a set of 'in-progress' transaction IDs

Taken at a point in time, it records which transactions were still running and therefore whose changes should stay invisible.

2 A row version is visible if...

Its creating transaction (xmin) committed before the snapshot was taken, and its deleting transaction (xmax), if any, had not committed.

3 Read Committed takes a new snapshot per statement

So each statement can see newly committed work from other transactions.

4 Repeatable Read / Serializable take one snapshot for the whole transaction

So the same data remains visible from the first query to the last.

Tuple Versions, Dead Tuples & Bloat

1 UPDATE = INSERT + mark-old-dead

An UPDATE never modifies a row in place; it writes a new tuple and marks the old one's xmax.

2 Dead tuples aren't reclaimed immediately

A row version becomes eligible for reuse only once no active snapshot could possibly still need it.

3 Table and index bloat

High-churn tables accumulate dead tuples faster than they're cleaned, inflating storage and slowing scans.

4 HOT updates reduce the cost

If an UPDATE doesn't touch indexed columns and there's room on the same page, PostgreSQL can avoid extra index entries entirely.

VACUUM and Autovacuum's Role

1 VACUUM reclaims dead tuple space

It scans a table, identifies dead row versions no snapshot can see anymore, and marks their space reusable.

2 Autovacuum runs automatically

A background process triggers VACUUM on tables once their dead-tuple ratio crosses a configurable threshold.

3 VACUUM also prevents transaction ID wraparound

Because XIDs are finite, VACUUM freezes old row versions so the 32-bit XID counter can safely wrap around.

4 VACUUM FULL rewrites the table

It reclaims all bloat and shrinks the file on disk, but takes an ACCESS EXCLUSIVE lock — usually reserved for maintenance windows.

Note: Long-running transactions hold back the 'oldest needed snapshot' — a common cause of runaway bloat in production.

MVCC by the Numbers

0

Locks for a Plain SELECT

Ordinary reads take no row locks at all — they rely entirely on snapshot visibility.

2

Hidden Version Columns

xmin and xmax alone determine whether a row version is visible to a given snapshot.

1

Snapshot per Read Committed Statement

Repeatable Read and Serializable instead use exactly one snapshot for the whole transaction.

MVCC Reads vs. Traditional Lock-Based Reads

MVCC (PostgreSQL's approach)

- Readers see a consistent snapshot
- Readers never block writers
- Writers never block readers
- Old versions need VACUUM cleanup
- Excellent read/write concurrency

Pure Lock-Based Concurrency

- Readers take shared locks on rows read
- Readers can block writers
- Writers can block readers
- No extra row versions to clean up
- Concurrency suffers under mixed workloads

9

Configuring Transaction Behavior

Setting isolation level, access mode, and session defaults

SET TRANSACTION ISOLATION LEVEL

Set the isolation level for the current transaction, or change the session-wide default:



```
BEGIN;  
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
-- ... queries ...  
COMMIT;  
  
-- Or in one line:  
BEGIN ISOLATION LEVEL REPEATABLE READ;  
  
-- Session-wide default for all future transactions:  
SET SESSION CHARACTERISTICS AS TRANSACTION  
ISOLATION LEVEL REPEATABLE READ;
```

SET TRANSACTION must be the very first statement after BEGIN, before any query touches data.

default_transaction_isolation (postgresql.conf or SET) controls the level for future transactions.

The level cannot be changed once a transaction has taken its first snapshot.

READ ONLY, DEFERRABLE, and Session Defaults

Additional transaction properties beyond isolation level:



-- Read-only transactions can enable extra optimizations

```
BEGIN READ ONLY;
```

-- DEFERRABLE + SERIALIZABLE + READ ONLY:

-- waits briefly to get a snapshot that can never abort

```
BEGIN ISOLATION LEVEL SERIALIZABLE, READ ONLY, DEFERRABLE;
```

```
SHOW default_transaction_isolation;
```

```
SHOW default_transaction_read_only;
```

READ ONLY rejects any INSERT/UPDATE/DELETE, which can reduce SSI's tracking overhead.

DEFERRABLE only matters for READ ONLY SERIALIZABLE transactions — it trades a short delay for zero abort risk.

Session defaults are commonly set per-role, e.g. reporting users default to READ ONLY.

10

Introduction to Locking

Why PostgreSQL still needs explicit locks alongside MVCC

Why Locking Is Still Needed Alongside MVCC

1 MVCC solves read/write conflicts, not write/write conflicts

Two transactions writing to the same row must still be serialized — MVCC alone can't decide who wins.

2 Some operations need exclusive access

Schema changes like `DROP TABLE` or `ALTER TABLE` must not run while other transactions are reading or writing that table.

3 Referential integrity needs coordination

Foreign key checks must ensure a referenced row isn't deleted while another transaction is inserting a row that references it.

4 Locks provide mutual exclusion where snapshots can't

Row and table locks are how PostgreSQL serializes the specific operations that truly must not overlap.

What MVCC Handles vs. What Locks Handle

Handled by MVCC (Snapshots)

- Plain SELECT queries
- Consistent multi-statement reads
- Readers not blocking writers
- Isolation-level guarantees
- Choosing which row version to see

Handled by Locks

- UPDATE / DELETE on the same row
- SELECT ... FOR UPDATE reservations
- Schema changes (DDL)
- Referential integrity checks
- Coordinating concurrent writers

PostgreSQL's Lock Granularities

1 Row-level locks

Held on individual rows, typically acquired by UPDATE, DELETE, and explicit SELECT ... FOR UPDATE/SHARE clauses.

2 Table-level locks

Held on entire tables, acquired automatically by nearly every SQL command, or explicitly via LOCK TABLE.

3 Page-level locks

Very short-lived internal locks used by the storage engine while reading or writing a data page — invisible to applications.

4 Advisory locks

Application-defined locks with no automatic connection to any table or row — pure cooperative locking for your own logic.

Note: This deck focuses on the two granularities application developers interact with directly: row-level and table-level locks.

11

Row-Level Locks

FOR UPDATE, FOR NO KEY UPDATE, FOR SHARE, FOR KEY SHARE

The Four Row Lock Modes

1 FOR UPDATE

The strongest row lock — reserves a row so no other transaction can update, delete, or lock it until this one ends.

2 FOR NO KEY UPDATE

Like FOR UPDATE, but permits concurrent FOR KEY SHARE locks; taken automatically by UPDATES that don't touch key columns.

3 FOR SHARE

Lets multiple transactions hold a shared read-lock on the same row, while blocking any UPDATE or DELETE of it.

4 FOR KEY SHARE

The weakest row lock — blocks only changes to the row's key columns; used internally by foreign key checks.

SELECT .. FOR UPDATE

Locks the selected rows against concurrent updates, deletes, and other row locks:



```
BEGIN;  
  SELECT * FROM accounts  
  WHERE id = 1  
  FOR UPDATE;  
  
-- Row is now locked; other FOR UPDATE / UPDATE / DELETE  
-- on this row will wait until this transaction ends.  
  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
COMMIT;
```

The classic 'reserve the row before I modify it' pattern.

Any other transaction's UPDATE, DELETE, or FOR UPDATE on that row blocks until COMMIT/ROLLBACK.

Plain SELECT without a locking clause is never blocked by FOR UPDATE.

SELECT ... FOR NO KEY UPDATE

A slightly weaker lock, used when you'll update non-key columns only:



```
BEGIN;  
  SELECT * FROM accounts  
  WHERE id = 1  
  FOR NO KEY UPDATE;  
  
  UPDATE accounts SET last_login = now() WHERE id = 1;  
COMMIT;  
  
-- Ordinary UPDATES take this lock automatically  
-- when they don't modify primary-key or unique columns.
```

Allows concurrent FOR KEY SHARE locks (e.g., a foreign key check) to proceed without waiting.

Still conflicts with FOR UPDATE, FOR NO KEY UPDATE, and FOR SHARE.

PostgreSQL chooses this mode automatically for most ordinary UPDATE statements.

SELECT ... FOR SHARE

Lets many readers lock a row together, while still blocking writers:



```
BEGIN;  
  SELECT * FROM products  
  WHERE id = 42  
  FOR SHARE;  
  
-- Other sessions can also take FOR SHARE / FOR KEY SHARE.  
-- But UPDATE or FOR UPDATE on this row must wait.  
COMMIT;
```

Useful when you must read a row and guarantee it won't change before you finish acting on it.

Multiple transactions can hold FOR SHARE on the same row simultaneously.

Common in 'read parent, then insert dependent child' referential patterns.

SELECT ... FOR KEY SHARE

The lightest row lock, protecting only the row's key from change:



```
BEGIN;  
  SELECT * FROM accounts  
  WHERE id = 1  
  FOR KEY SHARE;  
  
-- Blocks: changes to id (the key)  
-- Allows: FOR NO KEY UPDATE, e.g. UPDATE balance  
COMMIT;  
  
-- PostgreSQL takes this lock automatically when a  
-- referencing foreign key row is inserted or updated.
```

Primarily used internally to enforce foreign key constraints efficiently.

Two FOR KEY SHARE locks, or a FOR KEY SHARE and a FOR NO KEY UPDATE, never conflict.

Rarely used directly in application code, but essential to understand FK behavior under load.

Row Lock Mode Comparison

Lock Mode	Blocks Key Changes	Blocks Non-Key Changes	Typical Source
FOR KEY SHARE	Yes	No	Foreign key checks
FOR SHARE	Yes	Yes	Explicit SELECT ... FOR SHARE
FOR NO KEY UPDATE	No*	Yes	Ordinary UPDATE statements
FOR UPDATE	Yes	Yes	Explicit SELECT ... FOR UPDATE

Note: *FOR NO KEY UPDATE still conflicts with another FOR NO KEY UPDATE or FOR UPDATE — it only permits concurrent FOR KEY SHARE.

NOWAIT — Fail Fast Instead of Waiting

Instead of blocking, immediately raise an error if the row is already locked:



```
BEGIN;  
  SELECT * FROM tickets  
  WHERE id = 501  
  FOR UPDATE NOWAIT;  
  -- ERROR: could not obtain lock on row  
  --      in relation "tickets"  
COMMIT;
```

Useful for UI actions where waiting would be a bad user experience — fail immediately and show 'currently being edited'.

The application must catch the `lock_not_available` error and respond gracefully.

Avoids piling up long-held connections stuck waiting on a busy row.

SKIP LOCKED — Building a Job Queue

A complete worker pattern: claim exactly one unlocked job per call, atomically:



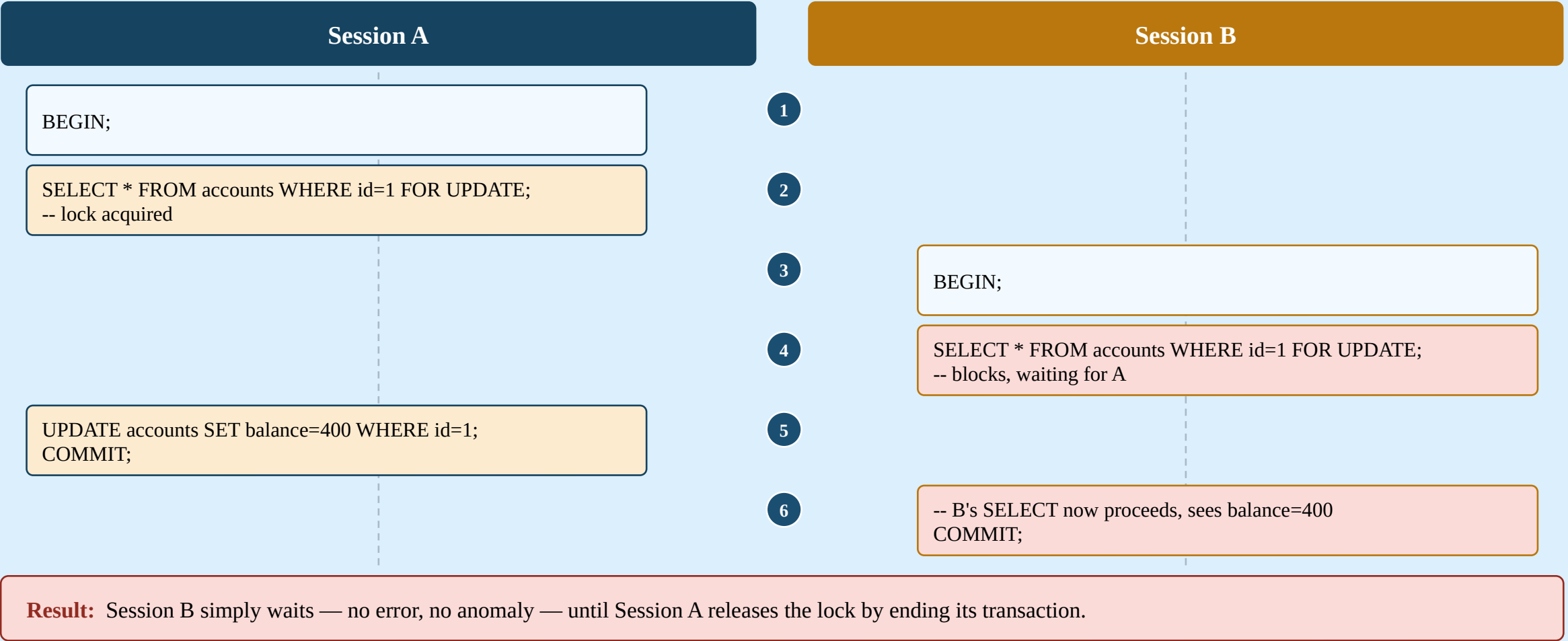
```
-- Run concurrently by any number of worker processes:
BEGIN;
WITH next_job AS (
  SELECT id FROM job_queue
  WHERE status = 'pending'
  ORDER BY priority DESC, id
  FOR UPDATE SKIP LOCKED
  LIMIT 1
)
UPDATE job_queue SET status = 'processing'
FROM next_job WHERE job_queue.id = next_job.id
RETURNING job_queue.*;
COMMIT;
```

Multiple worker processes can run this exact query concurrently without ever colliding.

Each worker grabs the next unlocked row instead of queuing up behind a busy one.

The CTE claims and updates the row atomically — no two workers can ever claim the same job.

Two Sessions Contending for FOR UPDATE



Row Locks and Foreign Keys

1 **FK checks take FOR KEY SHARE locks**

Inserting a row that references a parent takes a FOR KEY SHARE lock on that parent row to guard against concurrent deletion.

2 **This can cause surprising contention**

Many concurrent inserts referencing the same parent row all queue up briefly on that parent's key-share lock.

3 **FOR NO KEY UPDATE avoids blocking FK inserts**

Because ordinary UPDATES take FOR NO KEY UPDATE, they don't conflict with the FOR KEY SHARE locks FK checks use.

4 **Indexing foreign key columns matters**

An unindexed FK column forces a full scan to check for dependent rows during a parent DELETE, magnifying lock hold time.

When to Reach for Explicit Row Locks

1 Read-then-write logic

Whenever you `SELECT` a value, compute something from it, then `UPDATE` — use `FOR UPDATE` to prevent a lost update.

2 Queue and reservation systems

`SKIP LOCKED` for job queues; `FOR UPDATE NOWAIT` for 'claim this seat/ticket' style reservations.

3 Prefer atomic single statements when possible

A single `'UPDATE ... SET qty = qty - 1 WHERE qty > 0'` often avoids the need for an explicit lock entirely.

4 Keep locked transactions short

The longer a row stays locked, the more other sessions queue up behind it — never lock a row and then wait on user input.

12

Table-Level Locks

The eight lock modes that guard PostgreSQL's tables

Overview — Eight Table Lock Modes

1 Acquired automatically

Almost every SQL command takes a table lock behind the scenes, from a lightweight ACCESS SHARE on SELECT to a full ACCESS EXCLUSIVE on DROP TABLE.

2 Ranked from weakest to strongest

ACCESS SHARE, ROW SHARE, ROW EXCLUSIVE, SHARE UPDATE EXCLUSIVE, SHARE, SHARE ROW EXCLUSIVE, EXCLUSIVE, ACCESS EXCLUSIVE.

3 Compatible modes can coexist

Many transactions can hold ACCESS SHARE on the same table at once — it only conflicts with ACCESS EXCLUSIVE.

4 Stronger modes conflict more broadly

ACCESS EXCLUSIVE conflicts with every other mode, including itself — it demands sole access to the table.

ACCESS SHARE vs. ROW SHARE

ACCESS SHARE

- Taken by: plain SELECT
- The weakest, most common lock
- Conflicts only with ACCESS EXCLUSIVE
- Any number of readers coexist freely
- Example: `SELECT * FROM orders;`

ROW SHARE

- Taken by: `SELECT ... FOR UPDATE / FOR SHARE`
- Signals intent to lock specific rows
- Conflicts with EXCLUSIVE, ACCESS EXCLUSIVE
- Multiple sessions can hold it concurrently
- Example: `SELECT * FROM orders FOR UPDATE;`

ROW EXCLUSIVE vs. SHARE UPDATE EXCLUSIVE

ROW EXCLUSIVE

- Taken by: UPDATE, DELETE, INSERT
- The standard 'I'm writing rows' lock
- Conflicts with SHARE and stronger modes
- Many writers can proceed concurrently
- Example: UPDATE accounts SET ...;

SHARE UPDATE EXCLUSIVE

- Taken by: VACUUM, ANALYZE, CREATE INDEX CONCURRENTLY
- Protects against other maintenance operations
- Does NOT conflict with ROW EXCLUSIVE
- Self-conflicting: only one at a time
- Example: VACUUM (ANALYZE) accounts;

SHARE vs. SHARE ROW EXCLUSIVE

SHARE

- Taken by: `CREATE INDEX` (non-concurrent)
- Allows concurrent reads
- Blocks any row writes (ROW EXCLUSIVE)
- Multiple SHARE locks coexist
- Example: `CREATE INDEX ON accounts(id);`

SHARE ROW EXCLUSIVE

- Taken by: `CREATE TRIGGER`, some `ALTER TABLE` forms
- Stronger than SHARE — protects against SHARE too
- Self-conflicting: only one session may hold it
- Still allows plain reads (ACCESS SHARE)
- Example: `CREATE TRIGGER trg ... ON accounts;`

EXCLUSIVE vs. ACCESS EXCLUSIVE

EXCLUSIVE

- Taken by: rare — some REFRESH MATERIALIZED VIEW forms
- Allows only ACCESS SHARE (plain reads) concurrently
- Blocks all writers and other lock-takers
- Rarely seen in everyday application code
- Rows can still be read, not written

ACCESS EXCLUSIVE

- Taken by: DROP TABLE, TRUNCATE, ALTER TABLE, VACUUM FULL
- The strongest lock — blocks absolutely everything
- Conflicts with every mode, including itself
- Even a plain SELECT must wait for it to release
- Example: ALTER TABLE accounts ADD COLUMN x int;

Which Command Takes Which Lock?

Command	Lock Mode Acquired
SELECT	ACCESS SHARE
SELECT ... FOR UPDATE / FOR SHARE	ROW SHARE
INSERT, UPDATE, DELETE	ROW EXCLUSIVE
VACUUM (non-full), ANALYZE, CREATE INDEX CONCURRENTLY	SHARE UPDATE EXCLUSIVE
CREATE INDEX (non-concurrent)	SHARE
CREATE TRIGGER, some ALTER TABLE variants	SHARE ROW EXCLUSIVE
DROP TABLE, TRUNCATE, ALTER TABLE (most forms), VACUUM FULL, REINDEX	ACCESS EXCLUSIVE

Full Table Lock Conflict Matrix

	AS	RS	RE	SUE	S	SRE	E	AE
AS	•	•	•	•	•	•	•	X
RS	•	•	•	•	•	•	X	X
RE	•	•	•	•	X	X	X	X
SUE	•	•	•	X	X	X	X	X
S	•	•	X	X	•	X	X	X
SRE	•	•	X	X	X	X	X	X
E	•	X	X	X	X	X	X	X
AE	X	X	X	X	X	X	X	X

X = conflicts (cannot be held concurrently)

• = compatible (can be held simultaneously)

AS=Access Share RS=Row Share RE=Row Exclusive SUE=Share Update Exclusive S=Share SRE=Share Row Exclusive E=Exclusive AE=Access Exclusive

The LOCK TABLE Statement

1 Syntax

`LOCK TABLE [ONLY] table_name [, ...] [IN lock_mode MODE] [NOWAIT];`

2 Explicit control over table-level locking

Lets you manually acquire any of the eight lock modes ahead of a batch of statements.

3 Only valid inside a transaction block

`LOCK TABLE` outside `BEGIN...COMMIT` locks and immediately unlocks — effectively useless.

4 Default mode is ACCESS EXCLUSIVE

If you omit `IN ... MODE`, PostgreSQL takes the strongest possible lock, so specify the weakest mode that fits your need.

Explicit LOCK TABLE Example

Taking a SHARE lock manually to allow reads but block writes during a bulk export:



```
BEGIN;  
LOCK TABLE accounts IN SHARE MODE;  
  
-- Concurrent SELECTs proceed normally.  
-- Any UPDATE/DELETE/INSERT on accounts now waits.  
  
COPY accounts TO '/exports/accounts_snapshot.csv' CSV;  
COMMIT;
```

Guarantees the export sees a perfectly frozen table without stopping read traffic.

Always release the lock quickly — hold it only for the duration of the export.

Prefer REPEATABLE READ + pg_dump-style snapshotting for most real export needs.

DDL and Table Locks — ALTER TABLE Pitfalls

1

Most ALTER TABLE forms need ACCESS EXCLUSIVE

Adding a column with a volatile default, changing a column type, or adding a constraint typically locks out all other access.

2

The lock queues behind existing activity

If a long-running query already holds ACCESS SHARE, your ALTER TABLE must wait — and then blocks every new query behind it.

3

Some forms are lock-light in modern PostgreSQL

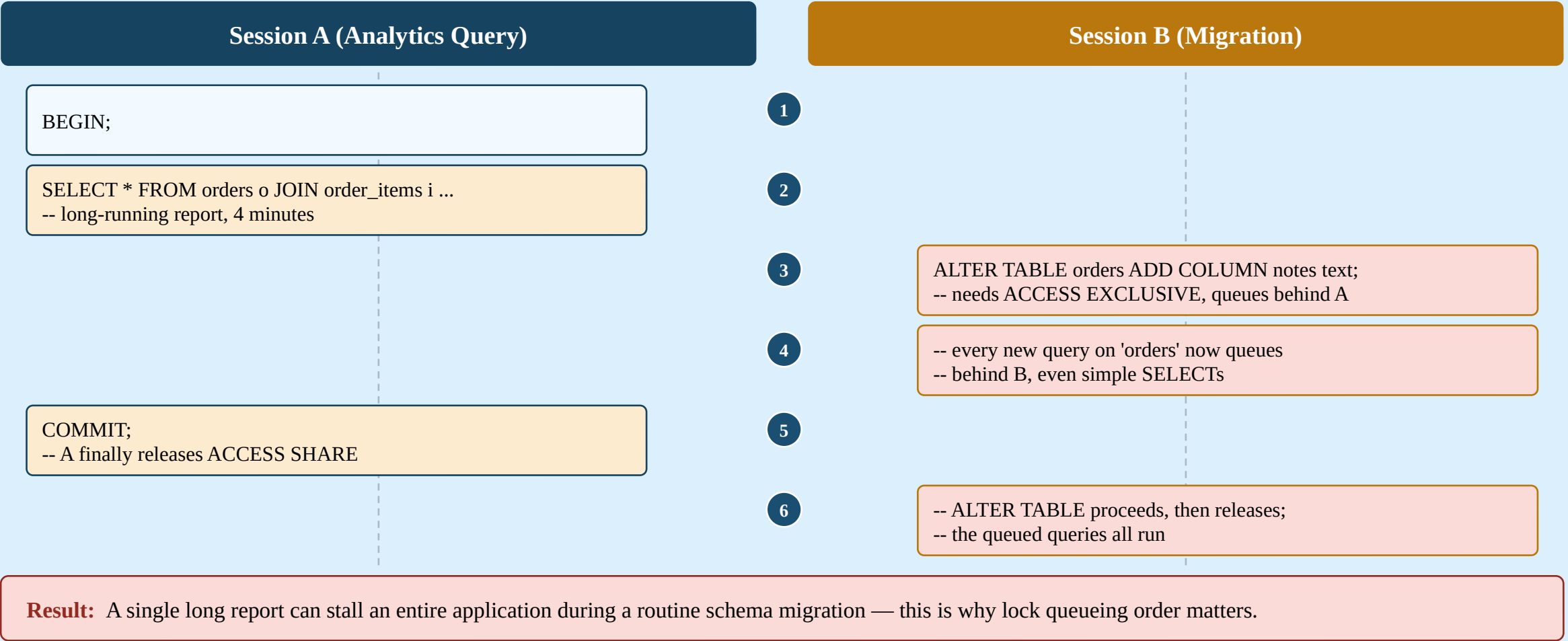
Adding a column with a constant default, or a nullable column with no default, only needs a brief metadata-only lock.

4

Always run DDL in a maintenance window or use CONCURRENTLY variants

For high-traffic tables, schedule migrations carefully and prefer non-blocking alternatives where they exist.

A Long Query Blocking an ALTER TABLE



Lock Queueing and Fairness (FIFO)

1 Locks are granted first-come, first-served

If a strong lock request is waiting, PostgreSQL queues subsequent weaker requests behind it rather than letting them jump ahead.

2 This prevents writer starvation

Without FIFO ordering, a constant stream of readers could indefinitely starve a writer waiting for a stronger lock.

3 But it means one slow ALTER TABLE can back up everything

Every request submitted after a blocked ACCESS EXCLUSIVE request also waits — even simple SELECTs.

4 lock_timeout lets you bound the wait

Setting lock_timeout aborts a statement that has waited too long for a lock, instead of queueing indefinitely.

Reducing Lock Contention on Hot Tables

1 Keep transactions short

The shorter a transaction, the shorter every lock it holds — never leave a transaction open while waiting on external I/O or user input.

2 Batch large writes

A single UPDATE touching 10 million rows holds its lock far longer than ten batches of 1 million, and blocks maintenance longer too.

3 Schedule heavy DDL for low-traffic windows

Or use CONCURRENTLY variants and lock_timeout with retries to avoid blocking production traffic.

4 Set statement_timeout and lock_timeout

Bound how long any single statement can run or wait, so one runaway query can't cascade into an outage.

CONCURRENTLY Variants

1 CREATE INDEX CONCURRENTLY

Builds an index without taking SHARE (which would block writes) — only SHARE UPDATE EXCLUSIVE, at the cost of a slower, two-pass build.

2 DROP INDEX CONCURRENTLY

Removes an index without an ACCESS EXCLUSIVE lock on the table, avoiding a stall on production traffic.

3 REINDEX CONCURRENTLY (PostgreSQL 12+)

Rebuilds an index in place, again avoiding the heavy lock a plain REINDEX would require.

4 The trade-off

CONCURRENTLY operations take longer, cannot run inside an explicit transaction block, and can fail partway, leaving an INVALID index to clean up.

CREATE INDEX CONCURRENTLY Example

Adding an index to a busy production table without blocking writers:



```
-- Must run outside an explicit transaction block:
CREATE INDEX CONCURRENTLY idx_orders_customer
ON orders (customer_id);

-- If it fails partway, check for an invalid index:
SELECT indexrelid::regclass
FROM pg_index WHERE NOT indisvalid;

-- Clean up and retry:
DROP INDEX CONCURRENTLY idx_orders_customer;
```

Takes SHARE UPDATE EXCLUSIVE, so INSERT/UPDATE/DELETE continue running normally.

Performs two full table scans instead of one, roughly doubling build time.

Always verify `pg_index.indisvalid` after a CONCURRENTLY build in automation scripts.

13

Advisory Locks

Application-defined locks that live outside any table or row

What Are Advisory Locks?

1 Locks with no automatic table or row connection

You choose an arbitrary integer (or pair of integers) as the lock key — PostgreSQL doesn't interpret its meaning at all.

2 Purely cooperative

Advisory locks only block other code that also explicitly checks the same lock key — they don't stop ordinary SQL on any table.

3 Fast and lightweight

Stored in a small in-memory hash table rather than the regular lock manager's per-row bookkeeping.

4 Two flavors

Session-level locks persist until explicitly released or the connection ends; transaction-level locks release automatically at COMMIT/ROLLBACK.

Session-Level Advisory Locks

Held until explicitly unlocked or the session disconnects — independent of any transaction:



```
SELECT pg_advisory_lock(12345);  
-- do exclusive work protected by key 12345  
SELECT pg_advisory_unlock(12345);  
  
-- Non-blocking variant:  
SELECT pg_try_advisory_lock(12345);  
-- returns true if acquired, false if already held
```

Great for 'only one instance of this batch job may run at a time' coordination.

Must be released explicitly — forgetting to unlock leaks the lock until disconnect.

`pg_try_advisory_lock` is ideal for a non-blocking 'am I the leader?' check.

Transaction-Level Advisory Locks

Automatically released at the end of the current transaction — no manual unlock needed:



```
BEGIN;  
  SELECT pg_advisory_xact_lock(67890);  
  -- protected work here  
COMMIT;  
-- lock is released automatically  
  
-- Non-blocking variant:  
SELECT pg_try_advisory_xact_lock(67890);
```

Safer default for most application code — no risk of a leaked lock from a forgotten unlock call.

Perfect for 'only one transaction may process this logical resource at a time' patterns.

Behaves like any other resource acquired during the transaction — released on COMMIT or ROLLBACK.

Advisory Lock Function Reference

Function	Scope	Behavior
pg_advisory_lock(key)	Session	Blocks until acquired
pg_try_advisory_lock(key)	Session	Returns immediately, true/false
pg_advisory_unlock(key)	Session	Releases one held lock
pg_advisory_xact_lock(key)	Transaction	Blocks until acquired, auto-released
pg_try_advisory_xact_lock(key)	Transaction	Non-blocking, auto-released
pg_advisory_unlock_all()	Session	Releases every session-level lock held

Real-World Advisory Lock Use Cases

1 Singleton batch jobs

Ensure a nightly report or cron-triggered job never runs twice concurrently, even if triggered by overlapping schedulers.

2 Leader election

Multiple application instances race for `pg_try_advisory_lock(1)`; whichever succeeds becomes the active leader.

3 Coordinating migrations

A deployment script takes an advisory lock before running schema migrations, preventing two deploys from racing.

4 Rate-limiting expensive operations

Serialize access to an external API or expensive computation across many application processes sharing one database.

14

Deadlocks

When two transactions wait on each other forever — and how PostgreSQL breaks the tie

What Is a Deadlock?

1 A circular wait

Transaction A holds a lock B needs, while B holds a lock A needs — neither can proceed, and neither will ever release its lock voluntarily.

2 Requires at least two resources and two transactions

The simplest case: two transactions, two rows, each locking one and then requesting the other, in opposite order.

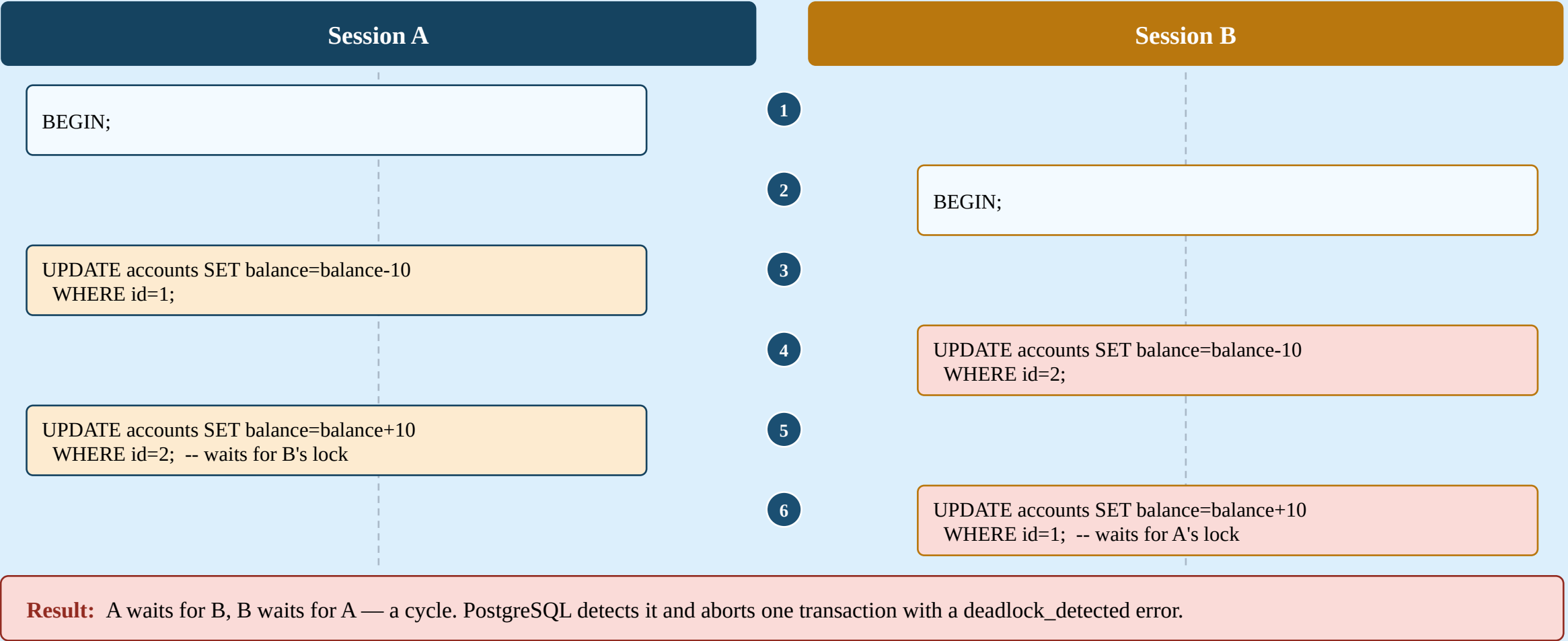
3 PostgreSQL detects it automatically

A background check periodically looks for wait cycles in the lock graph and breaks them by force.

4 Not the same as an ordinary lock wait

A normal wait resolves on its own once the blocking transaction ends — a deadlock never resolves without intervention.

Anatomy of a Classic Deadlock



How PostgreSQL Detects Deadlocks

A background check runs after every lock wait exceeds `deadlock_timeout`:



```
-- postgresql.conf  
deadlock_timeout = '1s' -- default
```

```
-- When a lock wait exceeds this, PostgreSQL builds the  
-- wait-for graph among all blocked backends and searches  
-- for a cycle. If found, it aborts one participant.
```

`deadlock_timeout` trades detection latency for overhead — checking too often wastes CPU on ordinary waits.

Deadlock checks are relatively expensive, which is why they only run after a wait, not on every lock request.

The victim transaction is typically the one whose abort breaks the cycle most cheaply.

Reading a Deadlock Error Message

PostgreSQL's deadlock error is deliberately detailed — read it closely:



```
ERROR: deadlock detected
DETAIL: Process 1234 waits for ShareLock on transaction 567;
        blocked by process 5678.
        Process 5678 waits for ShareLock on transaction 890;
        blocked by process 1234.
HINT: See server log for query details.
CONTEXT: while updating tuple (0,7) in relation "accounts"
```

The DETAIL line spells out the exact wait cycle between backend processes.

The victim's client receives SQLSTATE 40P01 — application code should catch this specifically.

log_lock_waits (covered in Section 15) adds even more context to the server log.

Deadlock Avoidance — Consistent Lock Ordering

1

The root cause is almost always inconsistent ordering

Deadlocks happen when different transactions lock the same set of rows or tables in different sequences.

2

The fix: always acquire locks in the same order

If every transaction updates accounts sorted by id, a cycle becomes structurally impossible.

3

Applies to tables too

Always touch table A before table B across your whole codebase, never the reverse in some code paths.

4

Document the convention

A team-wide rule ('always lock rows in ascending primary-key order') prevents deadlocks introduced by future code changes.

Lock Ordering Fix Example

Sorting rows before locking eliminates the earlier deadlock entirely:



```
-- Both sessions now always touch the lower id first:  
BEGIN;  
  UPDATE accounts SET balance = balance - 10  
    WHERE id = LEAST(:from_id, :to_id);  
  UPDATE accounts SET balance = balance + 10  
    WHERE id = GREATEST(:from_id, :to_id);  
COMMIT;
```

Every transfer now locks the smaller id first,
regardless of transfer direction.

Two concurrent transfers between the same two
accounts can never form a wait cycle.

This pattern generalizes to any 'lock a known set of
rows' operation.

Handling Deadlocks in Application Code

1 Deadlocks are expected, not exceptional failures

Well-designed applications treat SQLSTATE 40P01 as a signal to retry, not a bug to alert on.

2 Retry with backoff

On catching a deadlock error, roll back and re-run the entire transaction from the start, ideally after a short randomized delay.

3 Keep transactions short and simple

Fewer locks held for less time means fewer opportunities for a wait cycle to form in the first place.

4 Combine with lock ordering

Ordering prevents most deadlocks; retry logic gracefully handles the rare ones that still occur under complex workloads.

Deadlocks vs. Ordinary Lock Waits

1 An ordinary lock wait is normal and temporary

Session B simply pauses until Session A commits or rolls back — no error, no intervention needed.

2 A deadlock is a wait that can never resolve itself

Because the wait forms a cycle, no participant will ever release what another needs — PostgreSQL must intervene.

3 Most blocking you'll see is not a deadlock

The overwhelming majority of production 'my query is stuck' incidents are ordinary lock waits, not deadlocks.

4 Diagnosing the difference

`pg_locks` and `pg_stat_activity` (Section 15) show whether a session is merely waiting or genuinely stuck.

Deadlock Prevention Checklist

Practice	Why It Helps
Lock rows/tables in a consistent order	Removes the possibility of a circular wait
Keep transactions short	Fewer locks held, for less time
Acquire all locks as early as possible	Reduces windows where a partial lock set can conflict
Use lower isolation levels where correctness allows	Fewer serialization conflicts to retry or deadlock on
Add retry logic for SQLSTATE 40P01	Turns rare deadlocks into a transparent, automatic retry
Monitor deadlock frequency in production	A rising trend usually points to a specific code path to fix

15

Monitoring & Diagnostics

Finding, understanding, and resolving lock contention in production

pg_locks — Inspecting Current Locks

The system view showing every lock currently held or awaited across the cluster:



```
SELECT pid, locktype, relation::regclass, mode,  
       granted, transactionid  
FROM pg_locks  
WHERE NOT granted;
```

```
-- Every waiting lock request, with the PID that's blocked  
-- and the exact mode it's requesting.
```

granted = false rows are exactly what's stuck waiting right now.

Join to pg_stat_activity on pid to see the actual SQL text behind each lock.

relation shows which table the lock applies to (NULL for advisory or transaction-id locks).

pg_stat_activity — What's Running Right Now

A live view of every active connection and its current query:



```
SELECT pid, state, wait_event_type, wait_event,  
       query_start, query  
FROM pg_stat_activity  
WHERE state != 'idle'  
ORDER BY query_start;
```

wait_event_type = 'Lock' means this session is stuck waiting on a lock right now.

query_start lets you spot suspiciously long-running transactions before they cause an incident.

state = 'idle in transaction' is a common, easily overlooked cause of held locks.

Finding the Blocking Session

PostgreSQL provides a built-in function to trace who's blocking whom:



```
SELECT pid, pg_blocking_pids(pid) AS blocked_by, query
FROM pg_stat_activity
WHERE cardinality(pg_blocking_pids(pid)) > 0;
```

```
-- pid | blocked_by |          query
-- ----+-----+-----
-- 20481 | {20417} | UPDATE accounts SET balance = ...
```

`pg_blocking_pids()` returns the array of PIDs directly blocking a given session.

Chase the chain: the blocker's own query may reveal what it's waiting on, too.

This is usually the first query to run when 'everything feels stuck'.

Terminating a Blocking Session

As a last resort, cancel the offending query or end the connection entirely:



```
-- Cancel just the running query, keep the connection:  
SELECT pg_cancel_backend(20417);  
  
-- Forcefully terminate the entire connection:  
SELECT pg_terminate_backend(20417);  
  
-- Always confirm you've identified the right pid first!
```

`pg_cancel_backend` is gentler — it interrupts the current statement, rolling back its transaction.

`pg_terminate_backend` closes the connection outright; use it when cancel doesn't work.

Both require appropriate privileges — superuser, or the same role that owns the target session.

Key Diagnostic Views Reference

View / Function	What It Shows
pg_locks	Every lock held or awaited, by PID and relation
pg_stat_activity	Every connection's current query, state, and wait event
pg_blocking_pids(pid)	Which PIDs are directly blocking a given session
pg_stat_all_tables	Row counts, dead tuples, vacuum/analyze history per table
pg_stat_activity.state = 'idle in transaction'	Sessions holding an open transaction without active work
pg_cancel_backend / pg_terminate_backend	Interrupt a query, or close a connection outright

log_lock_waits and Logging Configuration

1 **log_lock_waits = on**

Logs a message whenever a session waits longer than `deadlock_timeout` for a lock — a cheap, high-value early warning system.

2 **log_min_duration_statement**

Logs any statement that takes longer than a threshold, surfacing the long transactions most likely to hold problematic locks.

3 **log_autovacuum_min_duration**

Tracks autovacuum runs that take unusually long, often a symptom of the same bloat that causes lock contention.

4 **Pair logs with monitoring dashboards**

Alert on rising lock-wait counts and deadlock frequency, not just on outright errors — contention is the earlier warning sign.

16

Best Practices & Wrap-Up

Bringing transactions, isolation, and locking together

Transaction Best Practices

1

Keep Them Short

- Never wait on I/O mid-tx
- Never wait on user input
- Commit as soon as work is done

2

Choose Isolation Deliberately

- Read Committed by default
- Escalate for named anomalies
- Test under real concurrency

3

Handle Errors Explicitly

- Use SAVEPOINT for partial undo
- Catch 40001 / 40P01
- Retry with backoff

4

Watch for Idle-in-Transaction

- It holds locks and snapshots
- Blocks VACUUM cleanup
- Set `idle_in_transaction_session_timeout`

Locking Best Practices

1

Lock in Consistent Order

Sort rows before locking
Same table order everywhere
Eliminates cycle formation

2

Use the Weakest Lock That Works

FOR KEY SHARE over FOR
UPDATE
SHARE UPDATE EXCLUSIVE for
maint.
Reduces blocking surface

3

Prefer CONCURRENTLY for DDL

CREATE INDEX
CONCURRENTLY
DROP INDEX CONCURRENTLY
Schedule ACCESS EXCLUSIVE
DDL

4

Monitor Continuously

pg_locks + pg_stat_activity
log_lock_waits = on
Alert on rising contention

Key Takeaways

1 Transactions are PostgreSQL's unit of correctness

BEGIN, COMMIT, ROLLBACK, and SAVEPOINT give you precise control over atomicity within a session.

2 MVCC is why PostgreSQL scales under concurrency

Readers and writers rarely block each other because every query sees a consistent snapshot, not a lock-guarded live table.

3 Isolation level is a deliberate trade-off

Read Committed for everyday work, Repeatable Read for consistent multi-step reads, Serializable when correctness cannot bend.

4 Locks are precision tools, not accidents

Understanding row vs. table lock modes — and monitoring `pg_locks` — turns mysterious 'hangs' into quickly diagnosed, fixable contention.

5 Tune timeouts, autovacuum, and indexes as a set

`statement_timeout`, `lock_timeout`, indexed foreign keys, and healthy autovacuum settings all work together to keep contention low.

Note: Thank you — practice these patterns against a real database, and every concept in this deck will become second nature.